

Edge Language Coach — Architecture Report

Scalable & Reliable Services

Contents

1. System Overview	1
2. Reliability	2
2.1 Circuit breaker — measured behaviour	2
2.2 Other primitives (summary)	3
3. Scalability	3
3.1 Mechanism	3
3.2 Measured scale-out behaviour	3
3.3 Threats to validity	4
4. Observability	4
4.1 Structured logging	4
4.2 Prometheus metrics	5
4.3 Dashboards & probes	5
5. Trade-offs	5
6. Future Work	6

1. System Overview

Edge Language Coach is an Italian-language learning application that uses LLM-driven conversational practice, automated topic curation from RSS feeds, and spaced-repetition flashcards. The system is deployed as three independently runnable units behind a load balancer, with a managed PostgreSQL backend (Supabase) and an external LLM provider (Groq).

Unit	Technology	Role
Web	React 19 + Vite, served by nginx	Single-page application, static build
Gateway	Fastify 5 (Node.js 22)	REST API, auth, reliability primitives
Workers	Node.js + BullMQ	Async job processing (scraper, summary, flashcard)

Unit	Technology	Role
Redis	redis:7-alpine	BullMQ queues + distributed rate-limit counters
Observability	Prometheus + Grafana	Metrics scraping and dashboards

The gateway is **stateless**: every request carries a Supabase JWT, every rate-limit counter lives in Redis, and no session state is held in process memory. This is the property that makes horizontal scale-out a one-line change (§3).

The request hot path:

Browser → web nginx (:80) → nginx-lb (:3001) → gateway:N → Redis | Supabase | Groq

The async path (long-running work):

gateway → Redis (BullMQ queue) → workers → Supabase

Async work is used for any operation that exceeds ~2 s of LLM time (post-session summary, flash-card pack generation, periodic topic scrape).

2. Reliability

Five primitives directly address the course’s reliability requirement, each tied to a specific failure mode.

Primitive	Code location	Failure addressed
Circuit breaker (opossum) on Groq	<code>apps/gateway/src/plugins/groq.ts</code>	Groq API slowdown / outage
Per-user rate limit (Redis-backed)	<code>apps/gateway/src/server.ts</code>	Abusive client flooding gateway
BullMQ retry + exponential backoff	<code>apps/workers/src/lib/queues.ts</code>	Transient worker / downstream failure
Dead-letter logging	<code>apps/workers/src/index.ts</code>	Job repeatedly failing past retry budget
Scraper idempotency key in Redis	<code>apps/workers/src/jobs/scrapeDuplicate.ts</code>	Duplicate work after worker restart

2.1 Circuit breaker — measured behaviour

The breaker was exercised end-to-end using a controllable mock Groq server (`mocks/groq-mock.mjs`) put into **slow mode** (30 s sleep per request, exceeding the 15 s breaker timeout) while a driver script (`mocks/breaker-driver.mjs`) fired 60 requests at concurrency 12. Full output preserved at `load/breaker-demo-output.txt`. Summary:

Phase	Calls	Per-call latency	Status
Closed (probing)	1 – 7	15.3 s (timing out against slow upstream)	breaker collecting failure samples
Transition	call 7	—	volumeThreshold (10) + 50 % error rate crossed → opened
Open (short-circuit)	8 – 60	12 – 58 ms	requests fail-fast, never reach upstream

Latency cliff: 15 300 ms → 25 ms in adjacent calls. Without the breaker, all 60 requests would have hit the 15 s timeout in parallel waves; with it, only 7 calls paid the upstream-stalled cost. The remaining 53 calls cost the system effectively zero CPU and zero open sockets to the failing upstream.

The breaker state is exposed as a Prometheus gauge (`groq_circuit_breaker_state`) so the closed → open → half-open transitions are visible in real time on the Grafana dashboard.

2.2 Other primitives (summary)

- **Rate limit:** 60 requests / minute per authenticated user, 60 / minute per IP otherwise. Counters live in Redis (`@fastify/rate-limit` Redis store) so the limit is shared across all gateway replicas without sticky sessions. Health probes are allow-listed.
- **Retries:** All BullMQ workers use `attempts: 3`, `backoff: { type: 'exponential', delay: 5000 }`. After 3 failed attempts the job is logged at `error` level with `dlq: true` and surfaces in the Bull Board UI at `:3002/queues`.
- **Idempotency:** The scraper sets a Redis key `scraper:slot:<date>:<6h-window>` with TTL = 6 h on success. A second invocation in the same window is a no-op, preventing duplicate topics if the job is replayed after a worker crash.

3. Scalability

The gateway scales horizontally; workers and Redis are vertically bounded but explicitly chosen as such (see §5).

3.1 Mechanism

```
docker compose up --scale gateway=3
```

An nginx load balancer (`nginx-lb`) sits in front of the gateway pool and distributes traffic via Docker’s built-in DNS round-robin. The web SPA and Prometheus both target `nginx-lb:3001`, so adding replicas requires no client-side or scrape-config change.

3.2 Measured scale-out behaviour

The k6 script at `load/gateway.js` runs three sequential scenarios in a single invocation:

Scenario	Profile
baseline	10 VUs constant for 30 s
stressed	Ramp 0 $\rightarrow$ 50 VUs over 60 s, then taper
scaled_out	50 VUs constant for 30 s

The full script was run twice — once with a single gateway replica, once with three replicas — holding all other parameters constant. Comparison:

Metric	1 replica	3 replicas	Δ
gateway_errors rate	6.81 %	0.27 %	−96 %
scaled_out p95 latency	2.43 s	2.15 s	−12 %
stressed p95 latency	1.73 s	1.84 s	+6 % (within noise)
Sustained throughput	39.3 req / s	39.4 req / s	unchanged

Interpretation. Adding replicas had a dramatic effect on the *error rate* (the 1-replica gateway dropped roughly 7 % of requests under saturation; with 3 replicas this fell to under 0.3 %), but only a modest effect on *latency* under saturation. Under sustained load:

- The **gateway itself** is the bottleneck for **availability** — a single Node event loop saturates under 50 concurrent VUs plus auth verification plus rate-limit checks, so it starts dropping connections.
- The **request hot path downstream** (Supabase RTT, Redis round-trips) is the bottleneck for **latency** — adding gateway replicas does not reduce per-request work outside the gateway.

The persistent 25 % `http_req_failed` rate seen in both runs is the rate-limiter doing its job correctly: 50 VUs $\times$ 10 req/s = 500 req/min vs. the configured 60 req/min cap. These are 429 responses, not failures of the system.

3.3 Threats to validity

- **Co-located load generator.** k6 ran on the same host as Docker Desktop, competing for CPU. Most visible as the unexpected **baseline** p95 regression on the 3-replica run.
- **Mocked Groq.** All Groq calls returned in < 1 ms, isolating the gateway/queue layer (the system under test) but making absolute p95 numbers unrepresentative of production.
- **Single iteration per configuration.** No statistical variance reported.

4. Observability

The implementation favours **metrics + structured logs** over distributed traces. The trade-off is documented (§5); the choice was deliberate to keep the demo scope tractable.

4.1 Structured logging

All services emit single-line JSON via `pino`. Every log line carries **service** (`gateway` / `workers`), **requestId** (propagated from an `X-Request-Id` header or generated), and the relevant domain identifiers (`sessionId`, `jobId`, `queue`). Log level is `LOG_LEVEL` env-tunable; the default in production is `info`.

4.2 Prometheus metrics

Both the gateway and the workers expose `/metrics` in Prometheus text format, scraped every 15 s. Key custom metrics:

Metric	Type	Labels	Where
<code>http_request_duration_seconds</code>	Histogram	method, route, status_code	gateway
<code>groq_request_duration_seconds</code>	Histogram	operation, status	gateway
<code>groq_circuit_breaker_state</code>	Gauge	breaker	gateway
<code>bullmq_job_duration_seconds</code>	Histogram	queue, status	workers
<code>jobs_dead_letter_total</code>	Counter	queue	workers
<code>queue_depth_total</code>	Gauge	queue	gateway, workers
<code>active_sessions_total</code>	Gauge	—	gateway

Node default metrics (event-loop lag, GC, RSS) are also collected via `collectDefaultMetrics`.

4.3 Dashboards & probes

A Grafana dashboard provisioned from `docker/grafana/dashboards/edge-coach.json` is auto-loaded on startup at `localhost:3000`. Panels: request rate by route, p50 / p95 / p99 latency, 5xx rate, circuit-breaker state, queue depth, active sessions.

Two health endpoints back container orchestration:

- `GET /livez` — always 200 (liveness signal).
- `GET /readyz` — pings Redis, Supabase, and Groq with 2 s timeouts; returns 503 with the failing dependency named in the body.

5. Trade-offs

Four explicit trade-offs are recorded as ADRs in the repository. The headline reasoning:

Decision	Alternative	Reason chosen
Node.js workers	Go microservice	Shared TypeScript types with the gateway; single toolchain; faster iteration
BullMQ + Redis	RabbitMQ / Kafka	One Redis instance for both queueing <i>and</i> rate limiting; lower operational surface for a course-scale system
Supabase	Self-hosted Postgres + bespoke auth	Eliminated multi-week auth implementation; trade-off is an external dependency

Decision	Alternative	Reason chosen
<code>prom-client</code> only	OpenTelemetry traces	Faster to instrument; trade-off is no cross-service tracing — accepted because the call graph is shallow (gateway → Groq, gateway → workers via queue)

The single most consequential trade-off is the last one: a richer submission would include OTel traces showing a request span propagated from gateway → BullMQ job → Groq. This is recorded as future work.

6. Future Work

- **OpenTelemetry traces.** Auto-instrument Fastify + ioredis + Prisma; manually carry trace context through the BullMQ job payload to span the full async path.
- **Worker autoscaling.** Workers are currently a fixed count; BullMQ already supports multiple consumers so this is wiring + manifest, not redesign.
- **Kubernetes manifests + HPA.** The compose setup is the demo target; production deployment would lift these into a Helm chart with a horizontal pod autoscaler driven by the existing `queue_depth_total` and `http_request_duration_seconds` metrics.
- **Statistical load-testing.** Multiple iterations per configuration with confidence intervals, run from a dedicated load-generation host (not co-located).

See the repository for full ADRs (`docs/adr/`), the Mermaid C4 diagrams (`docs/c4-diagram.md`), the SLO table (`docs/slo-table.md`), and the raw k6 / breaker outputs (`load/`).